Pipeline Data Processor Architecture

UML Class Diagram & Technical Design Document | Software Architecture Task

1. Executive Summary & Design Scope

In data streaming and batch ingestion pipelines, data processing engines must decouple runtime configuration from physical execution. This architecture specifies an extensible, decoupled processor capable of dynamic mode switching and multi-database persistence without conditional branching antipatterns (`switch/if-else` cascades).

The architecture fulfills all system requirements:

- **Pre-existing Domain Contracts**: Employs `Datapoint` (data abstraction), `ModeIdentifier` (enum: `DUMP`, `PASSTHROUGH`, `VALIDATE`), and `DatabaseIdentifier` (enum: `POSTGRES`, `REDIS`, `ELASTIC`).
- **Dynamic Reconfiguration**: Implements `configure(mode: ModeIdentifier, db: DatabaseIdentifier)` to change modes and swap databases at runtime.
- **Polymorphic Processing**: Implements `process(data: Datapoint)` whose execution behavior adapts dynamically based on the active mode and target database driver.
- **Abstract Database Interoperability**: Standardizes `insert(data)` and `validate(data)` across Postgres, Redis, and Elasticsearch backends.

2. Structural UML Class Diagram

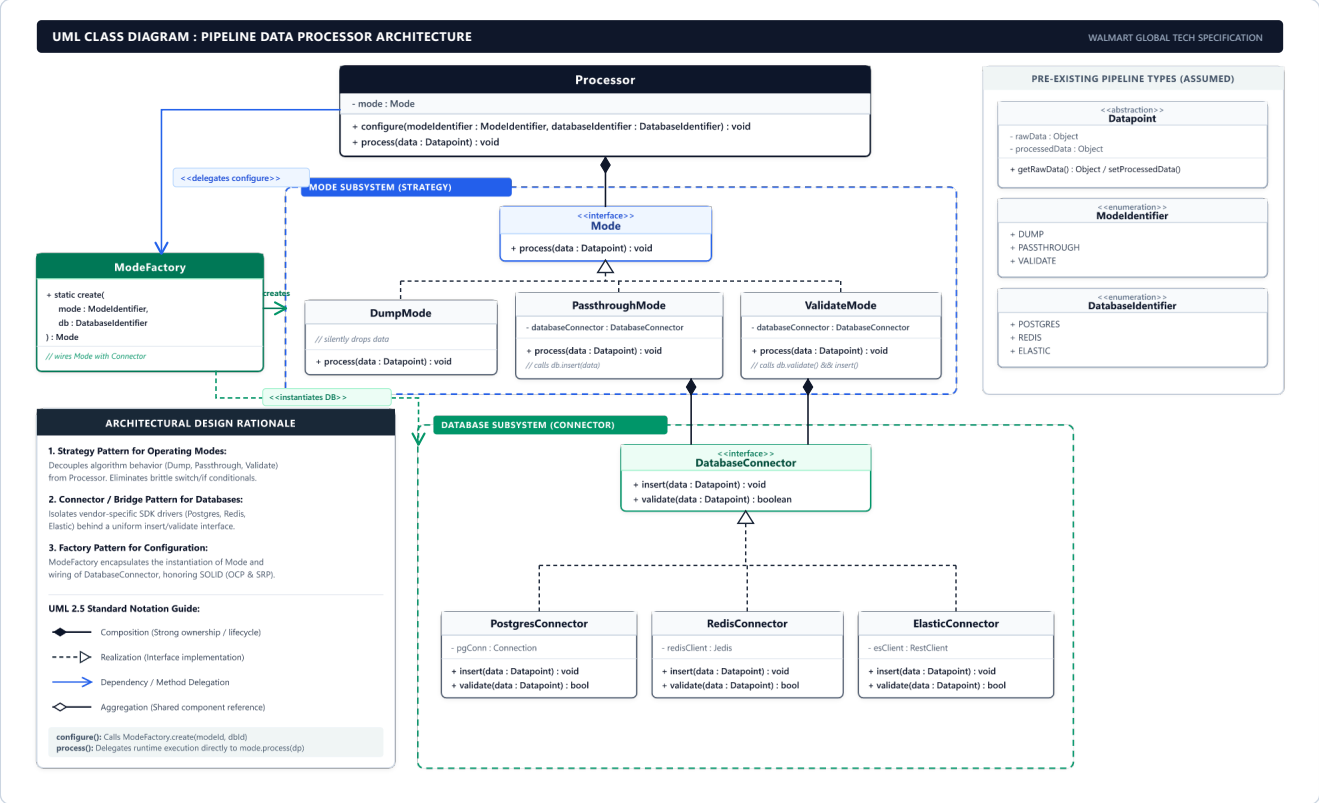


Figure 1: Complete UML 2.5 Class Diagram showing Strategy, Factory, and Connector hierarchies.

3. Architectural Decomposition & Pattern Allocation

The architecture leverages three fundamental Gang of Four (GoF) design patterns to achieve complete decoupling:

Pattern	Component	Architectural Rationale
Strategy Pattern	Mode Hierarchy	Encapsulates mode algorithms (`DumpMode`, `PassthroughMode`, `ValidateMode`) behind the `Mode` interface, isolating behavioral variations from the client.
Bridge / Connector	Database Subsystem	Decouples storage operations (`insert`, `validate`) from persistence vendors (`Postgres`, `Redis`, `Elastic`), allowing connectors to vary independently.

Factory Pattern	`ModeFactory`	Centralizes dynamic instantiation and dependency wiring during `configure()` calls, ensuring `Processor` remains free of construction logic.

3.1 Class & Interface Specifications

1. **`Processor`**:
   - **Attributes**: `- mode : Mode` (active strategy reference).
   - **Operations**: `+ configure(modeIdentifier : ModeIdentifier, databaseIdentifier : DatabaseIdentifier) : void`, `+ process(data : Datapoint) : void`
   - **Responsibility**: Context coordinator. Delegates construction to `ModeFactory` and data routing to `mode.process(data)`.
2. **`<<interface>> Mode`**:
   - **Operations**: `+ process(data : Datapoint) : void`
   - **Concrete Strategies**:
     - `DumpMode`: Silently drops incoming `Datapoint` instances (no-op).
     - `PassthroughMode`: Holds `- databaseConnector : DatabaseConnector` and invokes `databaseConnector.insert(data)`.
     - `ValidateMode`: Holds `- databaseConnector : DatabaseConnector`, evaluates `databaseConnector.validate(data)`, and invokes `insert(data)` only upon successful validation.
3. **`<<interface>> DatabaseConnector`**:
   - **Operations**: `+ insert(data : Datapoint) : void`, `+ validate(data : Datapoint) : boolean`
   - **Concrete Connectors**:
     - `PostgresConnector`: Implements relational schema validation and SQL insertion.
     - `RedisConnector`: Implements key-value validation and low-latency cache writes.
     - `ElasticConnector`: Implements document structure validation and Lucene inverted-index ingestion.
4. **`ModeFactory`**:
   - **Operations**: `+ static create(modeIdentifier : ModeIdentifier, databaseIdentifier : DatabaseIdentifier) : Mode`
   - **Responsibility**: Maps enum parameters to configured `Mode` instances with wired `DatabaseConnector` dependencies.

3.2 Dynamic Execution Lifecycle

```
1. Dynamic Reconfiguration:
   Client -> Processor.configure(ModeIdentifier.VALIDATE, DatabaseIdentifier.POSTGRES)
     Processor -> ModeFactory.create(VALIDATE, POSTGRES)
       ModeFactory -> instantiates PostgresConnector
       ModeFactory -> instantiates ValidateMode(postgresConnector)
     Processor updates internal reference: this.mode = configuredMode

2. Ingestion & Polymorphic Processing:
   Input Ingester -> Processor.process(datapoint)
     Processor -> activeMode.process(datapoint)  [Polymorphic dispatch]
       ValidateMode -> postgresConnector.validate(datapoint)
       If valid: ValidateMode -> postgresConnector.insert(datapoint)
```

4. SOLID Principles & Engineering Quality Audit

SOLID Principle	Status	Implementation Evidence in Diagram
Single Responsibility (SRP)	Compliant	`Processor` coordinates dispatch; `ModeFactory` creates instances; `Mode` executes pipeline rules; `DatabaseConnector` manages storage.
Open/Closed (OCP)	Compliant	Adding a new mode (e.g., `AuditMode`) or database (e.g., `MongoConnector`) requires zero changes to `Processor` or existing connectors.
Liskov Substitution (LSP)	Compliant	Any `Mode` or `DatabaseConnector` implementation can substitute for its interface without altering client correctness.
Interface Segregation (ISP)	Compliant	`DatabaseConnector` exposes only `insert` and `validate`; `Mode` exposes only `process`. Interfaces remain cohesive and lean.
Dependency Inversion (DIP)	Compliant	`Processor` and `Mode` implementations depend entirely on abstractions (`Mode`, `DatabaseConnector`), never on concrete drivers.

5. Production Reference Implementation (Java Architecture)

```java
public class Processor {
    private Mode mode;
    public void configure(ModeIdentifier modeId, DatabaseIdentifier dbId) {
        this.mode = ModeFactory.create(modeId, dbId);
    }
```

```
    public void process(Datapoint data) {
        if (this.mode != null) { this.mode.process(data); }
    }
}

public class ModeFactory {
    public static Mode create(ModeIdentifier modeId, DatabaseIdentifier dbId) {
        DatabaseConnector connector = resolveConnector(dbId);
        switch (modeId) {
            case DUMP:        return new DumpMode();
            case PASSTHROUGH: return new PassthroughMode(connector);
            case VALIDATE:    return new ValidateMode(connector);
            default:          throw new IllegalArgumentException("Unknown mode: " + modeId);
        }
    }
    private static DatabaseConnector resolveConnector(DatabaseIdentifier dbId) {
        if (dbId == null) return null;
        switch (dbId) {
            case POSTGRES: return new PostgresConnector();
            case REDIS:    return new RedisConnector();
            case ELASTIC:  return new ElasticConnector();
            default:       throw new IllegalArgumentException("Unknown database: " + dbId);
        }
    }
}

public class DumpMode implements Mode {
    public void process(Datapoint data) { /* Drops data silently */ }
}
public class PassthroughMode implements Mode {
    private final DatabaseConnector databaseConnector;
    public PassthroughMode(DatabaseConnector connector) { this.databaseConnector = connector; }
    public void process(Datapoint data) { this.databaseConnector.insert(data); }
}
public class ValidateMode implements Mode {
    private final DatabaseConnector databaseConnector;
    public ValidateMode(DatabaseConnector connector) { this.databaseConnector = connector; }
    public void process(Datapoint data) {
        if (this.databaseConnector.validate(data)) { this.databaseConnector.insert(data); }
    }
}
```

— End of Architecture Specification —